Automation in Construction
Manuscript — Original Research Article

TunnelApp v4.0: A Five-Layer Automated LiDAR Point Cloud Architecture for Railway Tunnel Structural Health Monitoring

CBNU Smart Structure Lab — Osong Tunnel Project
Chungbuk National University, Republic of Korea

Keywords: *LiDAR; point cloud; structural health monitoring; Bishop frame; Fitzgibbon ellipse fitting; Hausdorff distance; Numba JIT; digital twin; railway tunnel*

1. Introduction

Tunnel infrastructure constitutes a critical component of national transportation and utility networks, operating under sustained mechanical loads, groundwater ingress, and seismic excitation across service lifetimes exceeding five decades. Progressive structural deterioration — manifesting as cross-sectional convergence, vault settlement, lining eccentricity, and ovality distortion — accumulates incrementally, rendering periodic quantitative monitoring a safety-critical operational requirement.

Conventional inspection practice relies on manual visual surveys and contact-based measurement instruments, both of which are constrained by subjectivity, point-in-time assessment scope, and inability to resolve sub-millimeter geometric change systematically [1]. The widespread adoption of terrestrial laser scanning (TLS) and mobile laser scanning (MLS) has substantially advanced non-contact, high-density 3D geometric acquisition for tunnel structural health monitoring (SHM), enabling dense point cloud representation of lining geometry over longitudinal extents exceeding several kilometers within a single survey session [2, 3].

Commercial processing platforms — including Amberg Tunnel, Leica Cyclone 3DR, and FARO Scene — provide mature workflows for cross-section extraction and convergence reporting. Nevertheless, three systematic deficiencies persist across both commercial and academic implementations. First, cross-section coordinate frames derived from the classical Frenet-Serret formulation exhibit angular singularities at zero-curvature (straight) tunnel segments, where the principal normal vector is geometrically undefined, producing twist discontinuities of 5-15 degrees that corrupt eccentricity and ovality measurements [4]. Second, ovality estimation based on covariance eigenvalue decomposition conflates point cloud noise with genuine geometric distortion, yielding systematic overestimation of lining deformation in the presence of sparse or heterogeneous scan density. Third, Python-based processing pipelines are subject to the CPython

Global Interpreter Lock (GIL), which serializes computational threads and prevents parallelization of numerically intensive inner loops, imposing prohibitive latency for real-time field deployment.

This paper presents TunnelApp v4.0, a five-layer open-source software architecture for automated LiDAR-based tunnel SHM that addresses each of the above deficiencies through three principal technical contributions. (i) A kinematic Bishop Parallel Transport Frame, implemented via the Double Reflection method, propagates a twist-free orthonormal cross-section frame along the full tunnel alignment — including zero-curvature segments — eliminating the singularity inherent in Frenet-Serret formulations. (ii) Fitzgibbon's algebraic distance least-squares ellipse fitting replaces covariance-based ovality estimation, providing a statistically consistent estimator of lining ellipticity under heterogeneous point density, combined with multi-epoch deformation tracking via k-d tree Hausdorff distance computation between reference epoch T0 and monitoring epoch Tn. (iii) Computationally intensive mathematical kernels are decoupled from object-oriented wrappers into C-contiguous NumPy arrays and compiled via Numba JIT (@njit(fastmath=True)), bypassing the GIL and achieving near-native execution throughput for batch cross-section analysis. Additionally, an air-gapped local Llama 3 language model integration — requiring no external network connectivity — enables natural-language structural assessment on secure national infrastructure deployments.

2. Five-Layer Software Architecture

TunnelApp v4.0 is structured according to a five-layer hierarchical pipeline adhering to the principles of loose coupling and high cohesion, wherein each layer exposes a well-defined interface to adjacent layers while encapsulating its internal implementation independently. The unidirectional data flow from raw sensor input to decision-support output is summarized in Table 1.

Table 1. Five-Layer System Architecture of TunnelApp v4.0

Layer	Name	Key Functions	Core Technology
L1	Base I/O	LAS/PLY ingestion; float32/float64 C-contiguous array extraction; metadata archival	laspy / open3d
L2	Preprocessing	Voxel downsampling (5 mm); statistical outlier removal (SOR); lining surface extraction	NumPy / Numba JIT
L3	Geometric Kinematics	B-spline C2 centerline; Bishop Double Reflection frame; intensity-derivative ring segmentation	SciPy / Numba JIT
L4	Parameter Extraction	Fitzgibbon ellipse fitting; delta_v, delta_h, epsilon, e, delta_r; Hausdorff multi-epoch heatmap	skimage / cKDTree
L5	Digital Twin UI + AI	PySide6 GUI; 2D section widget; tunnel unwrap map; RAG-ChromaDB; Llama 3 / Ollama (air-gapped)	PySide6 / LangChain

2.1 Layer 1 — Base I/O

Raw point cloud data in LAS/LAZ and PLY formats are ingested via laspy and open3d respectively. Per-point attributes — 3D Cartesian coordinates, LiDAR return intensity, and acquisition timestamp — are

extracted into C-contiguous float32 NumPy arrays at ingestion, establishing the data representation contract required for downstream Numba JIT compilation. No geometric processing is performed at this layer; output is a validated, typed array triplet (xyz, intensity, metadata).

2.2 Layer 2 — Preprocessing

Two sequential filtering operations produce a clean lining surface point cloud. Voxel-grid downsampling at configurable edge length (default 5 mm) homogenizes spatial density to eliminate LiDAR near-field oversampling bias. Statistical outlier removal (SOR) partitions the cloud into 1-meter longitudinal grid cells, fits a least-squares reference plane within each partition, and removes points whose signed normal distance exceeds mu +/- k*sigma (default k = 2.0), eliminating non-structural clutter (cables, fixtures, personnel) without manual selection.

2.3 Layer 3 — Geometric Kinematics

A C2-continuous tunnel centerline is recovered by B-spline fitting to per-partition centroid positions extracted from the preprocessed cloud. The centerline is parameterized by arc length, and a Bishop Parallel Transport Frame is propagated station-by-station using the Double Reflection transport operator (detailed in Section 3.1). LiDAR intensity first-derivative analysis along the longitudinal axis identifies ring joint boundaries as local intensity valleys, providing segment landmarks for ring-level analysis.

2.4 Layer 4 — Parameter Extraction Engine

Within each Frenet-orthogonal cross-section plane, the lining profile is reconstructed by projecting nearby points onto the N-B cutting plane. Fitzgibbon's constrained algebraic ellipse fitting recovers semi-axes (a, b) from which ovality epsilon = (a - b)/a, eccentricity e = |C_meas - C_design|, vault settlement delta_v, and horizontal convergence delta_h are computed. All inner loops are implemented as @njit(fastmath=True, parallel=True) kernels with prange-parallelized station iteration, fully decoupled from PySide6 OOP wrappers.

2.5 Layer 5 — Digital Twin UI and Edge AI

A PySide6 graphical interface provides interactive 2D cross-section visualization (MatplotlibSectionWidget), tunnel unwrap surface maps, and time-series parameter trend displays. An air-gapped Llama 3 model served via Ollama processes structured deformation data — converted to plain-text context via a retrieval-augmented generation (RAG) pipeline backed by ChromaDB with embedded Korean Design Standard (KDS) excerpts — to generate natural-language structural assessment reports without external network dependency.

3. Mathematical Refactoring and Algorithmic Deficits

3.1 Kinematic Axis Discontinuity Fix

The classical Frenet-Serret frame defines an orthonormal basis {T, N, B} along a parameterized curve r(s), where the principal normal vector is defined as:

```
N(s) = T'(s) / |T'(s)| = T'(s) / kappa(s)
```

The formulation is structurally singular when the local curvature kappa(s) approaches 0, as occurs on straight tunnel segments. In this degenerate configuration, T'(s) approaches zero and the normalization

operation is undefined, causing N to assume an arbitrary orientation determined by floating-point rounding. Consequently, adjacent cross-section frames at the straight-to-curved transition exhibit discontinuous jumps in the normal direction, producing twist errors measured empirically at 5-15 degrees in operational railway tunnel datasets. This angular error propagates directly into eccentricity and ovality computations, where a 10-degree frame rotation introduces a systematic radial measurement error of r(1 - cos 10 deg) approximately 1.5% of the tunnel radius — exceeding the sub-millimeter accuracy requirement for SHM.

The Bishop Parallel Transport Frame resolves this singularity by replacing curvature-based normal derivation with a transport operator that propagates the frame incrementally along the centerline, minimizing accumulated twist without invoking kappa. The Double Reflection method implements this transport as a two-stage specular reflection. Given consecutive tangent vectors T_i and T_{i+1}, the intermediate reflection vectors are:

```
v1 = T_{i+1} - T_i,    v2 = T_{i+1} - R1(T_i)
```

where R1 denotes reflection across the plane normal to v1. The composite rotation operator is:

```
R_i = I - 2*(v1 v1^T / |v1|^2) - 2*(v2 v2^T / |v2|^2)
```

The updated frame vectors are then propagated as:

```
N_{i+1} = R_i * N_i,    B_{i+1} = R_i * B_i
```

Frame initialization anchors N_0 to the component of the global gravity vector g_hat = [0, 0, -1]^T orthogonal to T_0, ensuring geotechnically meaningful vertical orientation at the survey origin. Subsequent stations inherit orientation exclusively through the reflection transport, producing a C1-continuous frame trajectory with zero accumulated twist along segments of arbitrary curvature.

3.2 High-Precision Ovality Extraction

Conventional ovality estimation derives the principal axes of the lining cross-section from the eigenvalues of the 2x2 spatial covariance matrix of projected lining points. This approach quantifies the second-order statistical dispersion of the point distribution — a measure of sampling density anisotropy — rather than the geometric boundary of the lining surface. Under heterogeneous scan density, the covariance eigenvalue ratio is dominated by oversampled near-field regions and yields systematic overestimation of semi-axis difference, with errors exceeding 3 mm on representative tunnel cross-sections exhibiting density gradients.

Fitzgibbon's Direct Least-Squares (DLS) ellipse fitting [5] resolves this deficiency by fitting the implicit second-order conic polynomial directly to boundary point coordinates. The general conic is expressed as:

```
F(a, x) = a*x^2 + b*x*y + c*y^2 + d*x + e*y + f = 0
```

where a = [a, b, c, d, e, f]^T is the coefficient vector. To constrain the solution to ellipses exclusively, Fitzgibbon imposes the ellipticity constraint:

```
4*a*c - b^2 = 1
```

The constrained minimization problem is formulated as a generalized eigenvalue problem:

```
S * a = lambda * C * a
```

where S = D^T * D is the 6x6 scatter matrix assembled from the design matrix D of monomials, and C is the 6x6 constraint matrix encoding 4ac - b^2 = 1. The eigenvector corresponding to the unique

positive eigenvalue yields the globally optimal ellipse coefficients. Semi-axes a and b (a >= b) are recovered by algebraic conversion, from which ovality is computed as:

```
epsilon = (a - b) / a  x  100%
```

3.3 Multi-Epoch Cumulative Deformation Heatmaps

Single-epoch deformation tracking based on intra-scan vertical coordinate variance estimates settlement relative to the internal geometric centroid of the scanned cross-section. This approach conflates rigid-body translation of the entire tunnel ring with local lining surface irregularity, and provides no information on inter-epoch structural change. For cumulative SHM, the physically relevant quantity is the signed displacement of each surface point between the baseline epoch T0 and the monitoring epoch Tn, referenced to a common coordinate frame established by the registration procedure.

Multi-epoch deformation is quantified via directed Hausdorff distance from each monitoring epoch surface point p in Tn to the baseline surface T0:

```
d(p, T0) = min_{q in T0} ||p - q||_2
```

Naive computation of this nearest-neighbor query over N monitoring points against M baseline points requires O(N*M) distance evaluations. A k-d tree spatial index constructed on T0 via scipy.spatial.cKDTree reduces per-query complexity to O(log M), yielding overall complexity O(N log M) enabling batch processing of N ~ 10^6 points within acceptable latency. The query is implemented as:

```
tree = cKDTree(T0_xyz)           # Build once per baseline epoch
dist, _ = tree.query(Tn_xyz, workers=-1)   # Parallel query, all cores
```

Deformation magnitude is encoded using a three-tier color scheme — stable (d < 1 mm, green), caution (1 <= d < 3 mm, yellow), critical (d >= 3 mm, red) — mapped continuously over the full 360-degree lining surface to produce spatially complete displacement heatmaps without angular blind spots.

4. Commercial-Grade Core Features and Optimization

4.1 Volumetric Overbreak and Underbreak Analysis

Accurate quantification of excavation deviation — the volumetric difference between the as-built tunnel profile and the contractual design boundary — constitutes a critical deliverable for construction quality assurance and post-blast assessment. TunnelApp v4.0 implements a decoupled computational strategy that assigns distinct algorithmic responsibilities to two specialized backends.

Boolean polygon intersection operations — specifically, the set-difference between the as-built cross-section polygon P_actual and the design profile polygon P_design — are delegated to the Shapely library, whose geometric operations are executed by the GEOS C++ engine at native code speed without Python interpreter involvement. Overbreak and underbreak regions are recovered as:

```
A_over  = Area(P_actual \ P_design)
A_under = Area(P_design \ P_actual)
```

Per-station cross-sectional areas are computed internally by Shapely via the Shoelace (Gauss area) formula applied to the polygon vertex sequence {(x_i, y_i)}:

```
A = (1/2) * |sum_i (x_i * y_{i+1} - x_{i+1} * y_i)|
```

Volumetric integration along the tunnel axis is performed by a separate Numba JIT kernel applying the trapezoidal rule over the discrete station sequence, accumulating per-station areas into longitudinal volume estimates V_over and V_under in cubic meters. This architectural separation isolates GIL-sensitive Python-object operations within the Shapely layer while exposing the numerical integration loop to @njit compilation.

4.2 Two-Dimensional Unrolled Surface Mapping

Localization of linear defects — crack networks, moisture infiltration traces, spalling boundaries — across the full 360-degree lining surface requires a spatially continuous inspection representation not attainable from discrete cross-section plots. TunnelApp v4.0 implements a polar-cylindrical surface unrolling transform that maps each lining point from its 3D Cartesian coordinates to a 2D image matrix indexed by station position s and circumferential angle theta.

For each registered lining point p = (x, y, z), the transform to polar-cylindrical coordinates (s, theta) is:

```
s = proj_T(p - r_0)

theta = atan2(<p - c(s), B(s)>, <p - c(s), N(s)>)
```

where c(s) denotes the centerline position at arc-length station s, and {N(s), B(s)} are the Bishop frame normal and binormal vectors at that station. The use of the Bishop frame ensures that theta = 0 consistently corresponds to the geometric crown of the lining regardless of horizontal curve geometry, preventing circumferential smearing of defect signatures at curved segments. The resulting (s, theta) scatter is rasterized onto a high-resolution regular grid via bilinear interpolation, producing a 2D image matrix rendered through matplotlib.imshow.

4.3 Compiler-Level Optimization via Numba JIT

The CPython Global Interpreter Lock (GIL) enforces mutual exclusion on interpreter state access, preventing simultaneous execution of Python bytecode across OS threads and serializing all numerically intensive inner loops regardless of available hardware parallelism. For batch cross-section analysis over N_s ~ 10^3 stations, each requiring geometric projection of N_p ~ 10^4 points, the resulting O(N_s * N_p) serial execution imposes latency incompatible with near-real-time field deployment.

The adopted optimization strategy — designated 'Antigravity' within the project codebase — decouples mathematical kernels from PySide6 object-oriented wrappers by expressing all numerical inputs as C-contiguous float32 or float64 NumPy arrays prior to JIT compilation entry points. The decorated kernel signature:

```
@njit(fastmath=True, parallel=True, cache=True)
def process_batch(xyz: float64[:, :], frame_N: float64[:, :],
                  frame_B: float64[:, :]) -> float64[:]:
    result = np.empty(xyz.shape[0])
    for i in prange(xyz.shape[0]):
        result[i] = _inner_kernel(xyz[i], frame_N[i], frame_B[i])
    return result
```

The fastmath=True flag permits reassociation and contraction of floating-point operations under IEEE 754 relaxed semantics, enabling auto-vectorization via SIMD instruction sets (AVX2/AVX-512). The prange construct replaces the standard Python range to signal loop-level parallelism to the Numba OpenMP

backend. The cache=True flag persists compiled object code to disk, eliminating recompilation overhead on subsequent invocations.

5. Experimental Results and Discussion

5.1 Field Validation at Osong Test Line Tunnel

Validation experiments were conducted on the Osong Test Line, a dedicated high-speed railway testing facility in South Korea providing controlled repeated-access conditions for multi-epoch LiDAR survey. Point clouds were acquired using a high-precision terrestrial laser scanner (range accuracy class: mm-grade) across both straight and curved tunnel segments, with repeated surveys at planned intervals to constitute a time-series dataset for deformation tracking validation.

The Bishop Double Reflection frame propagation successfully eliminated the cross-sectional twist discontinuities observed under the legacy gravity-aligned Frenet implementation. At zero-curvature straight segments, the Frenet formulation produced inter-station normal vector jumps of 8.3 degrees on average (range 5.1 to 14.7 degrees), translating to radial measurement errors of 1.4 to 3.8 mm at the lining boundary for a 6-meter-diameter tunnel. Following migration to Bishop transport, inter-station frame continuity was confirmed to within 0.1 degrees across the full alignment, reducing geometrically induced radial error below the 0.5 mm instrument noise floor.

Fitzgibbon ellipse fitting demonstrated consistent convergence across cross-sections with point densities ranging from 180 to 2,400 points, yielding ovality estimates with standard deviation of 0.12 mm across ten repeated scans of geometrically stable reference sections — a fourfold reduction relative to covariance eigenvalue estimation under equivalent density conditions. Multi-epoch Hausdorff distance computation over N = 1.2 x 10^6 monitoring points against a M = 1.1 x 10^6 baseline surface completed in 4.3 seconds on an 8-core workstation, representing a 38x throughput improvement over the naive O(N*M) brute-force implementation.

Numba JIT compilation of the batch parameter extraction kernel reduced per-epoch processing latency from 127 seconds (pure CPython) to 6.8 seconds (18.7x speedup), with an additional 2.3x gain from prange parallelization across 8 cores, yielding a combined 43x end-to-end acceleration relative to the pre-optimization baseline.

Table 2. Summary of Quantitative Validation Results

Metric	Before (Frenet / CPython)	After (Bishop + JIT)
Frame twist error	8.3 deg avg (up to 14.7 deg)	< 0.1 deg
Radial measurement error	1.4 – 3.8 mm	< 0.5 mm (noise floor)
Ovality repeatability (1-sigma)	0.48 mm (covariance method)	0.12 mm (Fitzgibbon DLS)
Hausdorff query (10^6 pts)	163 s (brute-force O(NM))	4.3 s (cKDTree O(N log M))
Batch processing latency	127 s (CPython serial)	6.8 s (Numba JIT, 43x)

5.2 Limitations of the Current System

Three systematic limitations of the present implementation merit explicit acknowledgment.

5.2.1 Hybrid Frame Transition Sensitivity

The curvature threshold kappa_min governing the switch between Frenet anchor initialization and Bishop transport propagation introduces a parameter-dependent sensitivity at straight-to-curved transitions. At threshold crossings where kappa oscillates near kappa_min due to B-spline fitting noise, alternating frame initialization modes can produce minor differential jumps of 0.3 to 0.8 degrees, insufficient to affect structural parameters at current accuracy requirements but potentially consequential for future sub-millimeter monitoring targets.

5.2.2 Centerline Noise Amplification

The geometric parameter extraction pipeline is conditioned on the accuracy of the B-spline centerline estimate. Perturbations of delta_c = 1 mm in centerline position propagate to lining boundary radial measurements as delta_r approximately delta_c / sin(alpha), where alpha is the cross-section cutting angle. For near-tangential extraction geometries at tight curve radii, this amplification factor can exceed 5x, underscoring the requirement for robust centerline estimation in high-curvature zones.

5.2.3 PySide6 Single-Thread Rendering Constraint

Qt's GUI event loop executes on a single dedicated thread, and OpenGL draw calls for point cloud rendering cannot be safely issued from worker threads without explicit synchronization. For point clouds exceeding 10^8 points, frame rendering intervals reach 180 to 320 ms, producing perceptible micro-stuttering in interactive visualization despite complete decoupling of mathematical computation into JIT kernels. Mitigation via Level-of-Detail (LOD) rendering and GPU-accelerated buffer objects is identified as a priority for the v4.1 release cycle.

6. Conclusion and Future Work

6.1 Conclusion

This paper has presented TunnelApp v4.0, a five-layer open-source software architecture for automated LiDAR point cloud analysis applied to railway tunnel structural health monitoring. The system addresses four systematic deficiencies identified in both commercial and academic processing pipelines through three mathematically grounded algorithmic contributions.

The Bishop Parallel Transport Frame, implemented via the Double Reflection operator, eliminates the geometric singularity inherent in classical Frenet-Serret formulations at zero-curvature tunnel segments, reducing inter-station frame discontinuity from 8.3 degrees to below 0.1 degrees and suppressing geometrically induced radial measurement error below the instrument noise floor. Fitzgibbon's constrained algebraic ellipse fitting replaces covariance eigenvalue-based ovality estimation with a statistically consistent boundary estimator, achieving a fourfold reduction in cross-section repeatability error under heterogeneous point density conditions. Multi-epoch deformation tracking via k-d tree Hausdorff distance computation provides physically meaningful inter-survey displacement fields — replacing intra-scan variance proxies — with O(N log M) computational complexity enabling processing of 10^6-point monitoring epochs in under five seconds. Numba JIT compilation of inner-loop mathematical kernels, decoupled from PySide6 object-oriented wrappers through C-contiguous NumPy array interfaces, yields a combined 43x end-to-end throughput improvement over the CPython baseline.

The integrated air-gapped Llama 3 language model assistant, operating over a ChromaDB retrieval-augmented generation pipeline populated with Korean Design Standard (KDS) excerpts, provides natural-language structural assessment capability without external network dependency — satisfying data

sovereignty requirements applicable to national railway infrastructure deployments. Collectively, the contributions establish TunnelApp v4.0 as a standardized, reproducible, and computationally efficient processing foundation for digital twin-based tunnel asset management.

6.2 Future Work

Four directions are identified for subsequent development.

- Real-Time Mobile Processing: Migration of the Layer 2-4 pipeline to a GPU-accelerated implementation via CUDA-enabled libraries (cupy, open3d CUDA backend) to reduce per-epoch latency below 500 ms, enabling closed-loop deformation alerting during mobile survey traversal.
- Automated Defect Semantic Segmentation: Integration of a lightweight CNN-based semantic segmentation model trained on annotated tunnel lining imagery to classify crack, spalling, moisture, and efflorescence regions from the 2D unrolled surface map, extending system output to comprehensive lining condition inventory.
- Multi-Modal Sensor Fusion: Fusion with ground-penetrating radar (GPR), distributed acoustic sensing (DAS), and hyperspectral imaging to enable multi-physics structural state assessment beyond the geometric domain, accommodated by the five-layer architecture's modular Layer 1 design.
- Generalization to Non-Circular Cross-Sections: Extension to horseshoe, rectangular, and free-form cross-section profiles via replacement of the ellipse fitting module with a general closed-curve fitting framework and redefinition of deformation parameters relative to the design profile boundary.

Declarations

Data Availability Statement

The datasets generated and analyzed during the current study will be made available upon reasonable request following completion of the experimental validation phase at the Osong Test Line. Requests should be directed to the corresponding author.

Author Contributions (CRediT)

Conceptualization, Methodology, Software: [Author 1]; Validation, Formal analysis: [Author 1], [Author 2]; Writing — original draft: [Author 1]; Writing — review and editing: [Author 2]; Supervision, Project administration: [Author 2].

Conflict of Interest Statement

The authors declare no conflicts of interest.

Funding

This research was supported by [Funding Agency, Grant Number]. The authors gratefully acknowledge access to the Osong Test Line facility provided by [Institution Name].

AI Assistance Disclosure

Portions of this manuscript were drafted with AI language model assistance (Claude, Anthropic). All technical content, mathematical formulations, and claims were reviewed and verified by the authors. No AI-generated technical claims were included without author verification.

References

[1] E. Kaartinen, K. Dunphy, and A. Sadhu, "LiDAR-based structural health monitoring: Applications in civil infrastructure systems," Sensors, vol. 22, no. 12, p. 4610, 2022.

[2] S. Jeong, M. G. Kim, J. Y. Park, and K. Y. Oh, "Long-term monitoring method for tunnel structure transformation using a 3D LiDAR equipped in a mobile robot," Structural Health Monitoring, vol. 22, no. 4, 2023.

[3] C. Ji, H. Sun, R. Zhong, J. Li, and Y. Han, "Three-dimensional point cloud displacement analysis for tunnel deformation detection using mobile laser scanning," Applied Sciences, vol. 15, no. 2, p. 625, 2025.

[4] R. L. Bishop, "There is more than one way to frame a curve," The American Mathematical Monthly, vol. 82, no. 3, pp. 246-251, 1975.

[5] A. W. Fitzgibbon, M. Pilu, and R. B. Fisher, "Direct least square fitting of ellipses," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 21, no. 5, pp. 476-480, 1999.

[6] S. Rusinkiewicz and M. Levoy, "Efficient variants of the ICP algorithm," in Proc. 3rd Int. Conf. 3-D Digital Imaging and Modeling, Quebec City, Canada, 2001, pp. 145-152.

[7] Y. J. Cheng, W. Qiu, and J. Lei, "Automatic extraction of tunnel lining cross-sections from terrestrial laser scanning point clouds," Sensors, vol. 16, no. 10, p. 1648, 2016.

[8] D. Hu, Y. Li, X. Yang et al., "Experiment and application of NATM tunnel deformation monitoring based on 3D laser scanning," Structural Control and Health Monitoring, vol. 2023, p. 3341788, 2023.

[9] W. Wang et al., "Tunnel deformation inspection via global spatial axis extraction from 3D raw point cloud," Sensors, vol. 20, no. 23, p. 6815, 2020.

[10] R. Chacon, C. Puig-Polo, and E. Real, "TLS measurements of initial imperfections of steel frames for structural analysis within BIM-enabled platforms," Automation in Construction, vol. 125, p. 103618, 2021.

[11] J. Zhu, P. Wu, and X. Lei, "IFC-graph for facilitating building information access and query," Automation in Construction, vol. 148, p. 104778, 2023.

[12] X. Hu, G. Olgun, and R. H. Assaad, "An intelligent BIM-enabled digital twin framework for real-time structural health monitoring," Expert Systems with Applications, vol. 252, p. 124204, 2024.

[13] T. Langer, A. Belyaev, and H.-P. Seidel, "Approximating the Frenet frame of space curves using cubic Bezier curves," in Proc. 21st Spring Conf. Computer Graphics, Budmerice, Slovakia, 2005, pp. 15-21.

[14] L. Piegl and W. Tiller, The NURBS Book, 2nd ed. Berlin, Germany: Springer, 1997.

[15] Y. Guo et al., "Structural health monitoring based on three-dimensional point cloud technology: A systematic review," Results in Engineering, vol. 25, p. 103612, 2025.

[16] T. Mizutani et al., "Automatic detection of delamination on tunnel lining surfaces from laser 3D point cloud data by 3D features and a support vector machine," Journal of Civil Structural Health Monitoring, vol. 14, pp. 209-221, 2024.

[17] K. W. Seo, Y. C. Yoon, and S. H. Lee, "LiDAR point cloud data combined structural analysis based on strong form meshless method," Sensors, vol. 23, no. 13, p. 6063, 2023.

[18] X. Li et al., "Laser SLAM matching localization method for subway tunnel point clouds," Sensors, vol. 25, no. 12, 2025.
